

2025年度 情報システム工学PBL AI-Othello B-1班

接戦を演出し最終的に勝利するAI-Othello

Minimax法を応用したBayes-Even法により評価値を定数に近づけるプログラム

1. はじめに

目的: 対戦相手の強さに関わらず, 接戦を演出したうえで終盤に勝利する

わざと負けるAIもあるかもしれない
圧勝しても面白くない



相手がどんな強さであっても接戦をしたい

2. アイディア

講義で学習したMinimax法を応用する

自分の手番 (Even)

スコアを最大化する手を選ぶ

⇒ スコアが任意の定数に最も近い手を選ぶ

相手の手番 (Bayes)

相手は相手にとって最良の手を選ぶ (スコアを最小化する手を選ぶ)

⇒ 相手は必ず最良の手をとるとは限らない

⇒ 相手の強さを推定して相手が最も取りうる手を選ぶ

3. Bayes-Even法

自分の手番 (Even)

目的：評価値が目標値に最も近づく手を選ぶ

1. ターン数から目標値を決定
2. 全合法手を仮定
3. その手を打った先を再帰的に探索
4. 目標値との差が最小の分岐を選択
5. その分岐の評価値を返す

相手の手番 (Bayes)

1. 相手の各手の選択確率を推定
2. 各手の先を再帰的に探索
3. 確率×評価値を加算
4. 期待値として1つの評価値を返す

処理の流れ（中盤）

次の手として選択可能な手をすべて列挙する
差が最小となる手を選択



各手について「その手を打った後の盤面」を生成し、
将来の評価値を予測



予測評価値と目標値の差を計算



差が最小となる手を選択

4. ベイズ推定によって相手の強さを推定する

モデル更新

相手がとった手 **move** , および相手の強さを表す連続値 β に対して, 以下の式でベイズ推定を行い, 事前分布 $p(\beta)$ と尤度 $P(\mathbf{move} | \beta)$ から, 事後分布 $p(\beta | \mathbf{move})$ を求める.

$$p(\beta | \mathbf{move}) = \frac{P(\mathbf{move} | \beta)}{\int_0^1 P(\mathbf{move} | \beta)p(\beta)} \cdot p(\beta)$$

各合法手に対してその手を選ぶうる可能性

相手の強さを β と仮定したときに相手が **move** を選ぶ確率 (尤度) を $p(\mathbf{move} | \beta)$ とする. 尚, $\sigma(\mathbf{x})$ を softmax関数とする. 各合法手の中で **move** を選ぶ確率を $p(\mathbf{move})$ とし, 尤度にモデルを乗じて求める.

$$p(\mathbf{move} | \beta) = \sigma(\text{score}(\mathbf{move}) \cdot p(\beta))$$

$$p(\mathbf{move}) = \sum_{\beta} p(\mathbf{move} | \beta) \cdot p(\beta)$$

5. 評価関数

- ・ 評価関数とは、どこに石があるかの評価指標を数値化したもの

$$bp * 1 + cs * 10 + mobility * 2$$

bp : 盤面位置(×1)

cs : 確定石(×10)

mobility : 行動可能数(×2)

- ・ 最終的に上の評価関数で計算し、値を返却する

参考文献：「盤面評価値を用いたオセロプログラム」岸本浩一
近畿大学理工学部情報学科

↓ 盤面上のそれぞれの位置の評価を可視化した図

	0	1	2	3	4	5	6	7
0	100	-20	10	5	5	10	-20	100
1	-20	-50	-2	-2	-2	-2	-50	-20
2	10	-2	0	0	0	0	-2	10
3	5	-2	0	0	0	0	-2	5
4	5	-2	0	0	0	0	-2	5
5	10	-2	0	0	0	0	-2	10
6	-20	-50	-2	-2	-2	-2	-50	-20
7	100	-20	10	5	5	10	-20	100

6. AI-Othelloの実行結果

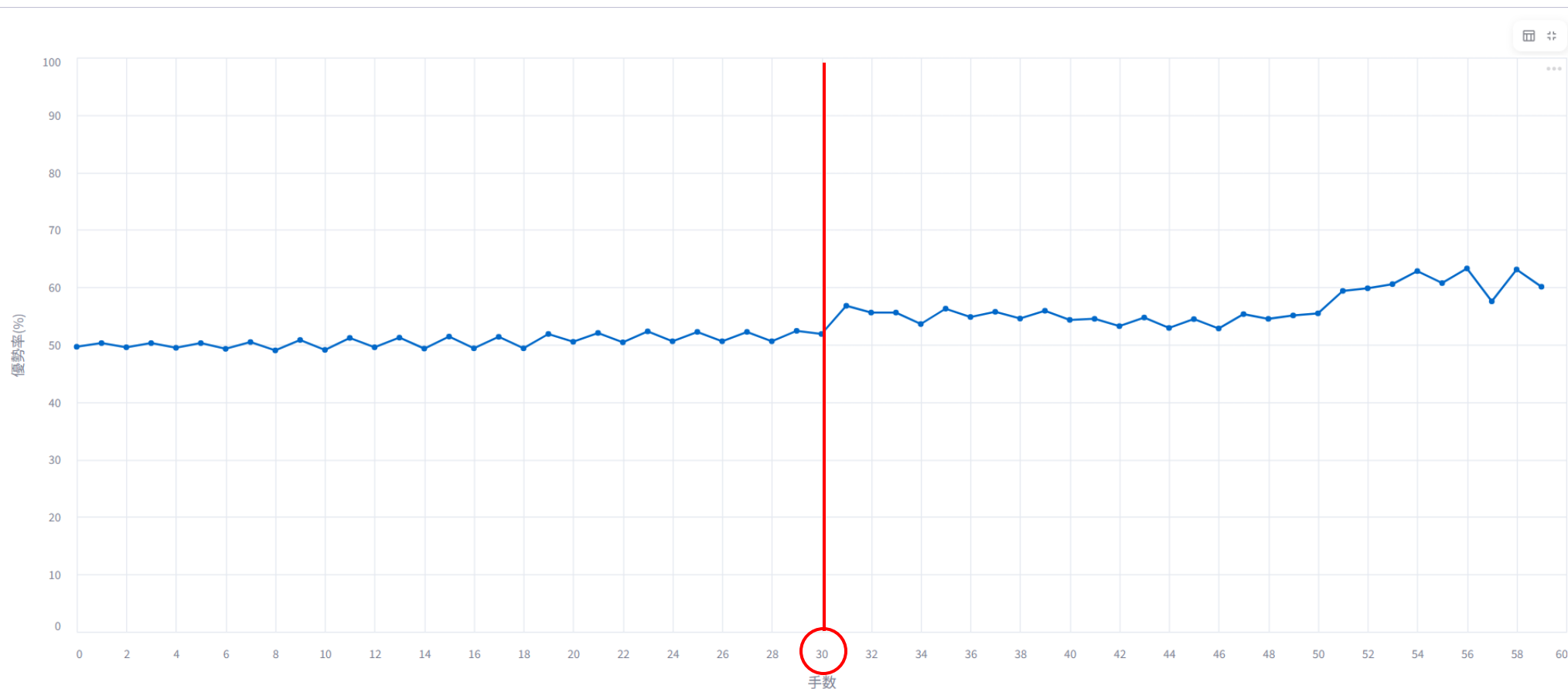


図 試合番号2-3 優勢率の変化（29手まで接戦）

7. 総括

- 考案したアルゴリズム bayes-even を用いたプログラムを実装し、プログラムを実行することができた.
- ベイズ推定によって相手の強さを推定することができた.
- 「盤面評価値を用いたオセロプログラム」を用いて評価関数を実装することができた.
- ライブラリ streamlit を用いて進捗やログを可視化することができた.
- matplotlib によってグラフを作成してアルゴリズムの有効性を検証できた.

課題

- 用いた評価関数・相手の強さのモデル β の事前分布が適切であったかどうかの検証
- 「深いコピー(deepcopy)」を繰り返したことによるメモリ負荷の増大
- 弱い相手が特に打ちやすい手, 強い相手が特に打ちやすい手を考慮したアルゴリズムの検討
- 相手が極めて強かった場合に即座にモデルを変更できるか

8. 参考文献

1. 石井健一郎・上田修功, “4・3 ベイズ推定”, “続・わかりやすいパターン認識 –教師なし学習入門–”, pp.52-55, (2014)
2. 赤石雅典, “7.5 softmax関数”, 安東一真, “最短コースでわかる PyTorch & 深層学習プログラミング”, pp.239-240, (2021)
3. Richard S. Sutton and Andrew G. Barto, “2.7 Gradient Bandits”, “Reinforcement Learning: An Introduction Second edition, in progress”, pp.42-43, (2014)
4. 岸本浩一, “盤面評価値を用いたオセロプログラム”, 近畿大学理工学部情報学科 卒業研究報告書, (2015)
5. Qiita @phd mech, “Pythonでゲーム作り：numpy, matplotlibを用いてオセロ（Othello）を実装”, https://qiita.com/phd_mech/items/e72d60ff51d7bbe7005c, 参照日: 2025.12.10
6. Future Coders KENICHIRO, “Streamlit入門 – 5) レイアウト”, <https://future-coders.net/streamlit%E5%85%A5%E9%96%80-5%E3%83%AC%E3%82%A4%E3%82%A2%E3%82%A6%E3%83%88/>, 参照日: 2025.12.10

補足(1) 定数によるAI-Othelloの調整

評価設計の目的

接戦を維持しつつ勝利するための評価値

本プログラムでは、オセロの進行段階に応じて評価値の目標を変化させる設計を採用した。

MAX_DEPTH : 探索の深さ

END_GAME : 終盤とみなすターン数

TARGET_MID : 中盤における目標評価値

TARGET__END : 終盤における目標評価値

DEPTH、DEPTH__MODEL、BETA_CANDIDATES : 相手の強さを評価する際に利用する

補足(2) softmax関数

softmax関数を用いて, スコアを確率分布に変換する. $\text{score}(\mathbf{move}) \cdot p(\beta)$ は, その手を評価し, 強さのモデルを乗じたものだが, 確率とはいえない. これを**preference**とし, 以下の式では $H_t(a)$ で表す.

$$\Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^n e^{H_t(b)}} = \pi_t(a)$$

言い換えれば, softmax関数にかけることで確率値として出力を得る. これは, 2値分類において, 1次関数の先にシグモイド関数があるのと同様である. softmax関数は次のような式でもあらわすことができる.

$$y_i = \frac{\exp(x_i)}{\sum_{k=1}^n \exp(x_k)} \quad (i = 1, 2, \dots, n)$$

補足(3) GitHubリポジトリ

分担して実装を行い, バージョン管理をするために, Git および GitHub を用いた.
実装中はprivateだったリポジトリをpublicにした.

GitHubリポジトリ: <https://github.com/miuruna/close-battle-othello-ai>

※README.mdなどのドキュメントがまだ十分ではない